Optimización Númerica - Tarea 2

Pablo Lorduy Montoya

(1,2) Método de bisección

El método de bisección busca una región donde posiblemente esté el mínimo de una función, evaluando el signo de la pendiente en los extremos del tramo. Para $f(x) = x^4 - 3x^2 + 1$ con $\frac{df(x)}{dx} = 4x^3 - 6x + 1$.

Con los scripts adjuntados. Se puede evidenciar que a mayor tolerancia, la diferencia entre el manual y el modelo de scipy es menor.

```
Iter1   a=-2.000   b=2.000   c=0.000   f(c)=0.000   df(c)=1.000   wide=4.000000
Iter2   a=-2.000   b=0.000   c=-1.000   f(c)=-3.000   df(c)=3.000   wide=2.000000
Iter3   a=-2.000   b=-1.000   c=-1.500   f(c)=-3.188   df(c)=-3.500   wide=1.000000
Iter4   a=-1.500   b=-1.000   c=-1.250   f(c)=-3.496   df(c)=0.688   wide=0.500000
Iter5   a=-1.500   b=-1.250   c=-1.375   f(c)=-3.472   df(c)=-1.148   wide=0.250000
Iter6   a=-1.375   b=-1.250   c=-1.312   f(c)=-3.513   df(c)=-0.169   wide=0.125000
Iter7   a=-1.312   b=-1.250   c=-1.281   f(c)=-3.511   df(c)=0.274   wide=0.062500
Iter8   a=-1.312   b=-1.281   c=-1.297   f(c)=-3.514   df(c)=0.056   wide=0.031250
Iter9   a=-1.312   b=-1.297   c=-1.305   f(c)=-3.514   df(c)=-0.055   wide=0.015625
Iter10   a=-1.305   b=-1.297   c=-1.301   f(c)=-3.514   df(c)=0.001   wide=0.007812
--
ERROR RESPECTO A SCIPY: 0.013463467326618657%
--

Iter1   a=-2.000   b=2.000   c=0.000   f(c)=0.000   df(c)=1.000   wide=4.000000
Iter2   a=-2.000   b=0.000   c=-1.000   f(c)=-3.000   df(c)=3.000   wide=2.000000
Iter3   a=-2.000   b=-1.000   c=-1.500   f(c)=-3.188   df(c)=-3.500   wide=1.000000
Iter4   a=-1.500   b=-1.000   c=-1.250   f(c)=-3.496   df(c)=0.688   wide=0.500000
Iter5   a=-1.500   b=-1.250   c=-1.375   f(c)=-3.472   df(c)=-1.148   wide=0.250000
Iter6   a=-1.375   b=-1.250   c=-1.312   f(c)=-3.513   df(c)=-0.169   wide=0.125000
Iter7   a=-1.312   b=-1.250   c=-1.281   f(c)=-3.511   df(c)=0.274   wide=0.062500
Iter8   a=-1.312   b=-1.281   c=-1.297   f(c)=-3.514   df(c)=0.056   wide=0.031250
Iter9   a=-1.312   b=-1.297   c=-1.305   f(c)=-3.514   df(c)=-0.055   wide=0.015625
Iter10   a=-1.305   b=-1.297   c=-1.301   f(c)=-3.514   df(c)=0.001   wide=0.007812
Iter11   a=-1.305   b=-1.301   c=-1.303   f(c)=-3.514   df(c)=-0.027   wide=0.003906
Iter12   a=-1.303   b=-1.301   c=-1.302   f(c)=-3.514   df(c)=-0.013   wide=0.001953
Iter13   a=-1.302   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=-0.006   wide=0.000977
Iter14   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=-0.003   wide=0.000488
Iter15   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=-0.001   wide=0.000244
Iter16   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=-0.000   wide=0.000122
Iter17   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=0.000   wide=0.000061
Iter18   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=0.000   wide=0.000031
Iter19   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=0.000   wide=0.000015
Iter20   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=0.000   wide=0.000008
Iter21   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=-0.000   wide=0.000004
```

```
Iter22   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=0.000   wide=0.000002
Iter23   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=-0.000   wide=0.000001
--
ERROR RESPECTO A SCIPY: 2.5754632624887088e-05%
--

Iter1   a=-2.000   b=2.000   c=0.000   f(c)=0.000   df(c)=1.000   wide=4.000000
Iter2   a=-2.000   b=0.000   c=-1.000   f(c)=-3.000   df(c)=3.000   wide=2.000000
Iter3   a=-2.000   b=-1.000   c=-1.500   f(c)=-3.188   df(c)=-3.500   wide=1.000000
Iter4   a=-1.500   b=-1.000   c=-1.250   f(c)=-3.496   df(c)=0.688   wide=0.500000
Iter5   a=-1.500   b=-1.250   c=-1.375   f(c)=-3.472   df(c)=-1.148   wide=0.250000
Iter6   a=-1.375   b=-1.250   c=-1.312   f(c)=-3.513   df(c)=-0.169   wide=0.125000
Iter7   a=-1.312   b=-1.250   c=-1.281   f(c)=-3.511   df(c)=0.274   wide=0.062500
Iter8   a=-1.312   b=-1.281   c=-1.297   f(c)=-3.514   df(c)=0.056   wide=0.031250
Iter9   a=-1.312   b=-1.297   c=-1.305   f(c)=-3.514   df(c)=-0.055   wide=0.015625
Iter10   a=-1.305   b=-1.297   c=-1.301   f(c)=-3.514   df(c)=0.001   wide=0.007812
Iter11   a=-1.305   b=-1.301   c=-1.303   f(c)=-3.514   df(c)=-0.027   wide=0.003906
Iter12   a=-1.303   b=-1.301   c=-1.302   f(c)=-3.514   df(c)=-0.013   wide=0.001953
Iter13   a=-1.302   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=-0.006   wide=0.000977
Iter14   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=-0.003   wide=0.000488
Iter15   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=-0.001   wide=0.000244
Iter16   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=-0.000   wide=0.000122
Iter17   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=0.000   wide=0.000061
Iter18   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=0.000   wide=0.000031
Iter19   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=0.000   wide=0.000015
Iter20   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=0.000   wide=0.000008
Iter21   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=-0.000   wide=0.000004
Iter22   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=0.000   wide=0.000002
Iter23   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=-0.000   wide=0.000001
Iter24   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=-0.000   wide=0.000000
Iter25   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=0.000   wide=0.000000
Iter26   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=-0.000   wide=0.000000
Iter27   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=-0.000   wide=0.000000
Iter28   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=0.000   wide=0.000000
Iter29   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=0.000   wide=0.000000
Iter30   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=-0.000   wide=0.000000
Iter31   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=-0.000   wide=0.000000
Iter32   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=-0.000   wide=0.000000
Iter33   a=-1.301   b=-1.301   c=-1.301   f(c)=-3.514   df(c)=-0.000   wide=0.000000
--
ERROR RESPECTO A SCIPY: 1.660355968906151e-08%
--
```

3. Grid

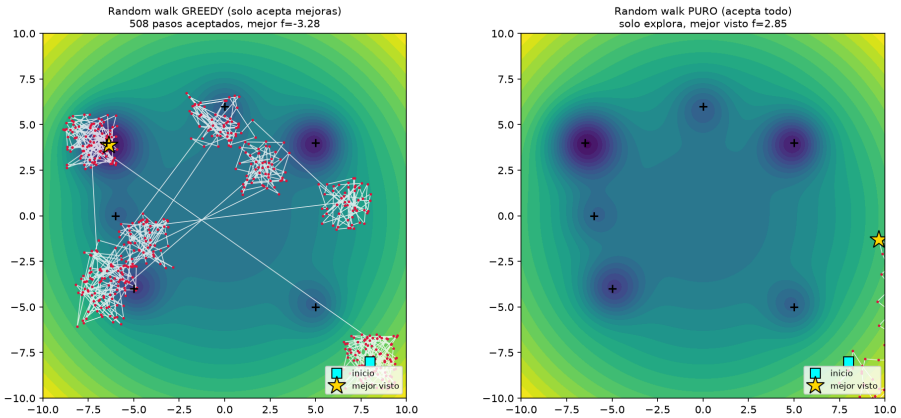
Para un número de dimensiones (D) de 2 tiempo de ejecución tomó aproximadamente de 0.003 segundos, mientras cuando D es 4 el tiempo se incrementa a un segundo completo. Esto demuestra que a mayor cantidad de dimensiones el costo computacional es mayor.

```
GRID
--
```

```
d = 4
[-2.49251098e-05  5.61958463e-05 -1.10763169e-05  1.87354404e-05]
Time taken: 0.7281336784362793
--
GRID
--
d = 2
[1.50118006e-05 5.49148258e-06]
Time taken: 0.0038335323333740234
---
Diferencia de tiempo D=4 a D=2: 0.724300 s
---
```

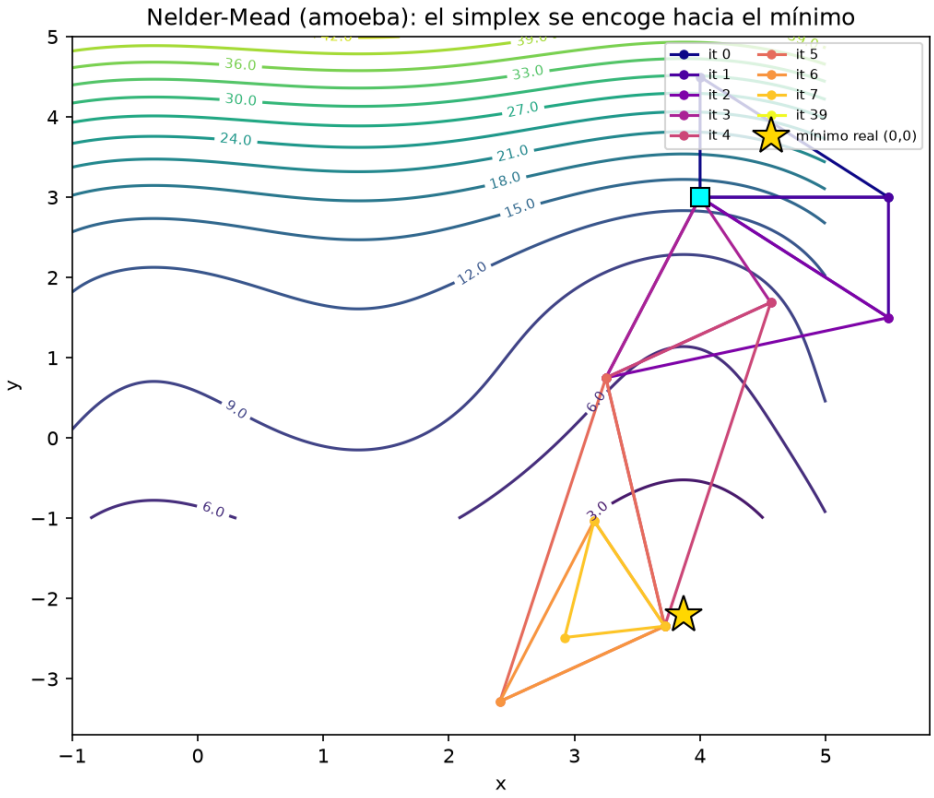
4. Random Walk

Agregando un reinicio en la caminata, se puede observar cómo la búsqueda se expande cerca de 3 valles permitiendo comparar entre 3 regiones diferentes donde se pueden ver posibles mínimos locales; al hacer un reinicio en un punto lejano de la última trayectoria permite que se explore más.



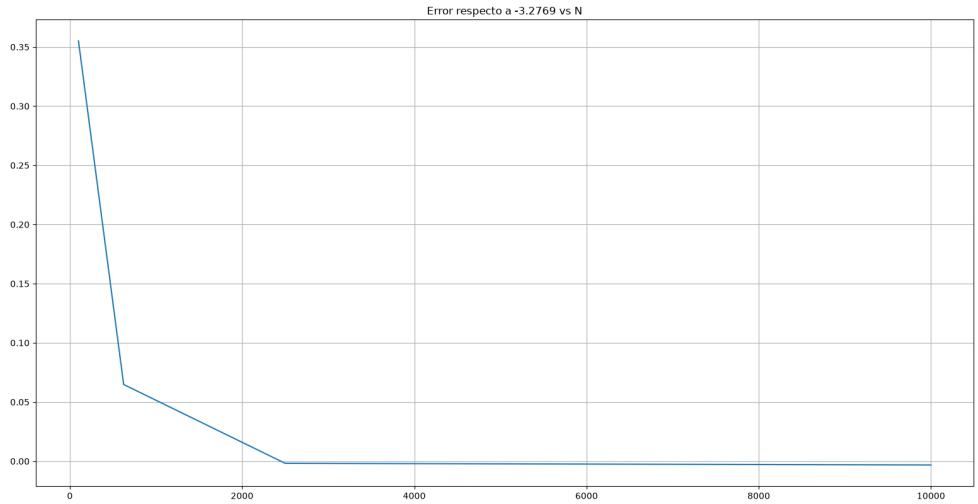
5. Nelder Mead

Aunque el punto real no se encuentre dentro del último simplex, se evidencia cómo se encoge hacia la curva de mínimo nivel estando muy cercano del mínimo encontrado. Teniendo en cuenta que el método manual y por medio de scipy se obtuvo el mismo valor.



6. Comparación del Error

A la hora de graficar el error vs el número de evaluaciones, se puede observar que el error tiende a reducir de la forma N^{-3} logarítmica; pues el número de evaluaciones de la forma que se relacione el costo de evaluaciones es N^d. Por lo que tiene sentido que al tener un costo mayor y una malla más densa, el porcentaje de error se reduzca.



7. Reflexión

El método de Nelder es más eficiente que un grid, con pocas evaluaciones porque hace una evaluación entre una región de 3 puntos, lo que hace que pueda llegar a converger cerca del

mínimo más rápido que un método de grid; el método de grid toma una malla cuasi uniforme, por lo que si la densidad de la malla no es suficiente, carecerá de precisión.

El método de Nelder pierde una ventaja operativa para funciones donde se presenten varios mínimos locales, pues el grid evalúa toda la región especificada; compara todos los puntos.